BPS-17 Lecturer Mathematics
AJKPSC / FPSC / PPSC / KPSC
Comprehensive MCQ Study Plan

Prepared by:

Zamir Hussain,
Lecturer Mathematics
University of Wah

Computing Tools for Mathematics

MATLAB **Maple** **Python** **Mathematica** **R** **Scilab**
GNU Octave

Topics Covered:

- Introduction & Overview of CAS
- MATLAB Programming & Functions
- Maple: CAS & Symbolic Algebra
- Mathematica: Notebooks & Kernels
- Python: NumPy & SciPy
- Python: Matplotlib & Visualization
- GNU Octave vs MATLAB
- Numerical Methods & CAS
- Data Analysis Tools
- MATLAB Basics & Matrix Operations
- Symbolic Computation in MATLAB
- Maple Procedures & Plots
- Mathematica: Pattern Matching
- Python: SymPy (Symbolic Math)
- R Language: Statistics & Plots
- Scilab: Open-Source CAS
- ODE & PDE Solvers
- Comparison & Applications

Each section: **Concept Summary → Key Commands → MCQs with Full Explanations**

Reference Material — PSC 2021–26

Contents

1. Introduction to Computing Tools for Mathematics

Computer Algebra Systems (CAS) are software packages that perform symbolic and numerical mathematical computations. They can:

- Simplify algebraic expressions symbolically
- Solve equations (algebraic, differential, integral)
- Perform numerical computations with high precision
- Visualize mathematical objects (2D, 3D plots)
- Automate repetitive mathematical tasks via programming

Major Tools:

Tool	Developer	Type
MATLAB	MathWorks	Numerical + Symbolic (Symbolic Toolbox)
Maple	Maplesoft	Primarily Symbolic CAS
Mathematica	Wolfram Research	Symbolic + Numerical CAS
Python (SciPy/SymPy)	Open-source	Numerical + Symbolic
R	R Foundation	Statistical Computing
GNU Octave	GNU Project	Free MATLAB clone (Numerical)
Scilab	ESI Group	Free MATLAB alternative

Key Commands & Syntax

Classification of Computing Tools:

- **CAS (Computer Algebra System):** Maple, Mathematica, Maxima — primarily symbolic
- **Numerical:** MATLAB, GNU Octave, Scilab — primarily numerical/matrix-based
- **Statistical:** R, SPSS, SAS — primarily data/statistics
- **General Purpose:** Python — numerical + symbolic via libraries
- **Proprietary:** MATLAB, Maple, Mathematica, Maple
- **Free/Open-source:** Python, R, GNU Octave, Scilab, Maxima, SageMath

1.1. MCQs: Introduction to Computing Tools

Q1. Which of the following is a **Computer Algebra System (CAS)** primarily designed for **symbolic computation**?

(A) MATLAB

(B) GNU Octave

(C) Maple

(D) Scilab

Correct Answer: (C) Maple
Maple (by Maplesoft) is a full-featured CAS primarily designed for symbolic computation — it handles algebra, calculus, differential equations, and more symbolically. MATLAB, GNU Octave, and Scilab are primarily numerical/matrix-based tools, though MATLAB can do symbolic math via its optional Symbolic Math Toolbox.

Q2. MATLAB is developed and maintained by which company?

(A) Wolfram Research

(B) Maplesoft

(C) MathWorks

(D) IBM

Correct Answer: (C) MathWorks
MATLAB (Matrix Laboratory) is developed by MathWorks, Inc. Wolfram Research develops Mathematica, and Maplesoft develops Maple.

Q3. Which of the following tools is **completely free and open-source** and is most compatible with MATLAB syntax?

(A) Scilab

(B) GNU Octave

(C) R

(D) Maple

Correct Answer: (B) GNU Octave
GNU Octave is free, open-source, and designed to be mostly compatible with MATLAB syntax. Most MATLAB scripts run with little or no modification in Octave. Scilab is also free but uses a different syntax.

Q4. Which Python library is used specifically for **symbolic mathematics** (analogous to Maple/Mathematica)?

(A) NumPy

(B) SciPy

(C) SymPy

(D) Pandas

Correct Answer: (C) SymPy
SymPy (Symbolic Python) is Python's library for symbolic mathematics — it can differentiate, integrate, solve equations, and simplify expressions symbolically. NumPy and SciPy are numerical libraries, while Pandas is for data manipulation.

Q5. The software **Mathematica** uses which proprietary programming language?

(A) MATLAB language

(B) Maple language

(C) Wolfram Language

(D) Julia

Correct Answer: (C) Wolfram Language
Mathematica is built on the **Wolfram Language**, a multi-paradigm symbolic computation language developed by Stephen Wolfram. It supports symbolic, numerical, and graphical computations in an integrated notebook environment.

FPSC Exam Tip: Remember the developer–product pairing:

MATLAB ↔ MathWorks
Maple ↔ Maplesoft
Mathematica ↔ Wolfram Research
R ↔ R Foundation
Python ↔ PSF (Python Software Foundation)

MATLAB's file extension is `.m`, Mathematica uses `.nb` (notebook), Maple uses `.mw` (worksheet).

2. MATLAB: Basics, Matrices, and Operations

MATLAB Fundamentals

MATLAB (Matrix Laboratory) is built around **matrices and arrays**. Key features:

- **Variables:** No type declaration needed; auto-assigned
- **Indexing:** 1-based (first element is `A(1)`, not `A(0)`)
- **Semicolon:** `;` suppresses output
- **Comments:** `%` symbol
- **Script files:** `.m` extension
- **Row vector:** `[1 2 3]` or `[1,2,3]`
- **Column vector:** `[1;2;3]`

- **Matrix:** `[1 2; 3 4]`

Key Commands & Syntax

Essential MATLAB Operations:

Command	Description
`zeros(m,n)`	$m \times n$ matrix of zeros
`ones(m,n)`	$m \times n$ matrix of ones
`eye(n)`	$n \times n$ identity matrix
`rand(m,n)`	$m \times n$ random matrix (uniform)
`randn(m,n)`	$m \times n$ random matrix (normal)
`size(A)`	Returns dimensions of matrix A
`length(v)`	Length of vector v
`numel(A)`	Total number of elements
`A'`	Transpose of A (conjugate transpose)
`A.*B`	Element-wise multiplication
`A*B`	Matrix multiplication
`det(A)`	Determinant of A
`inv(A)`	Inverse of A
`eig(A)`	Eigenvalues/eigenvectors of A
`linsolve(A,b)`	Solve $Ax = b$

2.1. MCQs: MATLAB Basics and Matrices

Q6. In MATLAB, what does the command `A = [1 2; 3 4]` create?

(A) A row vector of 4 elements

(B) A 2×2 matrix

(C) A 4×1 column vector

(D) A cell array

Correct Answer: (B) A 2×2 matrix
In MATLAB, spaces (or commas) separate elements in the *same row*, while semicolons `;` start a new row. So `[1 2; 3 4]` creates the matrix $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$.

Q7. What is the MATLAB command to create a 3×3 **identity matrix**?

(A) `ones(3)`

(B) `zeros(3,3)`

(C) `eye(3)`

(D) `diag(3)`

Correct Answer: (C) `eye(3)`
`eye(n)` creates an $n \times n$ identity matrix (1s on the main diagonal, 0s elsewhere). `ones(3)` creates a 3×3 matrix of all ones; `zeros(3,3)` creates all zeros.

Q8. In MATLAB, which operator performs **element-wise multiplication** of two arrays?

(A) `*`

(B) `.*`

(C) `^`

(D) `.^`

Correct Answer: (B) `.*`
In MATLAB, `.*` performs element-wise (pointwise) multiplication. The `*` operator performs standard matrix multiplication. Similarly, `./` is element-wise division and `.^` is element-wise exponentiation.

Q9. What is the output of `size([1 2 3; 4 5 6])` in MATLAB?

(A) `[3, 2]`

(B) `[2, 3]`

(C) `6`

(D) `[1, 6]`

Correct Answer: (B) `[2, 3]`
The matrix `[1 2 3; 4 5 6]` has 2 rows and 3 columns. `size(A)` returns `[rows, cols]`, so the output is `[2, 3]`.

Q10. In MATLAB, what does the **semicolon** `;` do at the end of a command?

(A) Terminates the program

(B) Starts a new block

(C) Suppresses the output from being printed

(D) Begins a comment

Correct Answer: (C) Suppresses the output from being printed
A semicolon at the end of a MATLAB statement suppresses output. Without `;`, MATLAB prints the result to the Command Window. The `%` symbol is used for comments.

Q11. In MATLAB, array/matrix indexing starts from:

(A) 0 (zero-based)
(B) 1 (one-based)
(C) -1
(D) Depends on data type

Correct Answer: (B) 1 (one-based)
MATLAB uses **1-based indexing** — the first element of array `A` is `A(1)`, not `A(0)`. This differs from C, Java, and Python which are 0-based.

Q12. Which MATLAB command solves the linear system $Ax = b$?
(A) `solve(A,b)`
(B) `A \ b`
(C) `A/b`
(D) `inv(A)*b` only

Correct Answer: (B) `A \ b` (backslash operator)
The backslash operator `A\b` solves the linear system $Ax = b$ using Gaussian elimination (LU decomposition). It is numerically more stable and efficient than computing `inv(A)*b`, though both give the same result for square non-singular A.

FPSC Exam Tip: MATLAB-specific things to remember:

- Indexing: MATLAB starts at 1; Python/C start at 0
- `A'` = conjugate transpose; `A.'` = simple transpose (no conjugation)
- `A*B` = matrix multiplication; `A.*B` = elementwise
- Colon operator: `1:5` = `[1 2 3 4 5]`; `0:0.5:2` = `[0, 0.5, 1, 1.5, 2]`

3. MATLAB: Programming, Control Flow, and Functions

MATLAB Programming Constructs

MATLAB supports full procedural programming:

- **For loop:** `for i = 1:n ... end`
- **While loop:** `while condition ... end`
- **If-else:** `if cond ... elseif cond ... else ... end`
- **Functions:** Defined in `.m` files; first line: `function [out] = fname(in)`

- **Anonymous functions:** `f = @(x) x^2 + 1`
- **Scripts vs Functions:** Scripts share workspace; functions have own workspace

3.1. MCQs: MATLAB Programming

Q13. Which of the following correctly defines an **anonymous function** in MATLAB that computes $f(x) = x^2 + 2x$?

(A) `f(x) = x^2 + 2*x`

(B) `function f = x^2 + 2*x`

(C) `f = @(x) x.^2 + 2.*x`

(D) `def f(x): return x**2 + 2*x`

Correct Answer: (C) `f = @(x) x.^2 + 2.*x`
MATLAB anonymous functions use the `@(args) expression` syntax. Option (D) is Python syntax. Options (A) and (B) are not valid MATLAB. Using `.^` and `.*` ensures the function works on arrays.

Q14. What is the output of the following MATLAB code?

```
s = 0; for i = 1:4 s = s + i; end disp(s)
```

(A) 4

(B) 10

(C) 24

(D) 16

Correct Answer: (B) 10
The loop computes $s = 1 + 2 + 3 + 4 = 10$. The variable `s` accumulates the sum of integers from 1 to 4.

Q15. In MATLAB, the `nargin` function inside a function returns:

(A) The number of output arguments

(B) The number of input arguments actually passed

(C) The total number of arguments defined

(D) The name of the function

Correct Answer: (B) The number of input arguments actually passed
`nargin` returns the number of input arguments passed by the caller. Similarly,

`nargout` returns the number of output arguments requested. These are commonly used to write functions with optional arguments.

Q16. Which MATLAB command is used to **plot** a 2D graph of $y = \sin(x)$?

(A) `draw(x, sin(x))`

(B) `graph(x, sin(x))`

(C) `plot(x, sin(x))`

(D) `show(x, sin(x))`

Correct Answer: (C) `plot(x, sin(x))`
`plot(x,y)` is MATLAB's basic 2D plotting command. For 3D, `surf`, `mesh`, or `plot3` are used. Common usage: `x = 0:0.01:2*pi; plot(x, sin(x))`.

Q17. In MATLAB, what does `fzero(@(x) x^3 - x - 2, 1)` compute?

(A) The minimum of $x^3 - x - 2$ near $x = 1$

(B) A root of $x^3 - x - 2$ near $x = 1$

(C) The derivative of $x^3 - x - 2$ at $x = 1$

(D) The definite integral of $x^3 - x - 2$ from 0 to 1

Correct Answer: (B) A root of $x^3 - x - 2$ near $x = 1$
`fzero(fun, x0)` finds a zero (root) of the function `fun` near the initial guess `x0`. For $f(x) = x^3 - x - 2$, the real root is approximately $x \approx 1.5214$.

4. Maple: Computer Algebra System

Maple is a powerful CAS by Maplesoft specializing in:

- **Symbolic algebra:** Factoring, expanding, simplifying
- **Calculus:** Exact differentiation, integration, limits, series
- **Differential equations:** `dsolve` for ODEs and PDEs
- **Linear algebra:** Exact matrix operations (via `LinearAlgebra` package)
- **Plotting:** 2D and 3D visualization
- **Procedures:** Programming via `proc ... end proc`

Maple uses **worksheet** (`.mw`) files and **document** (`.maple`) files.

Key Commands & Syntax

Essential Maple Commands:

Command	Description
`diff(f, x)`	Differentiate f with respect to x
`int(f, x)`	Indefinite integral of f w.r.t. x
`int(f, x=a..b)`	Definite integral $\int_a^b f\,dx$
`limit(f, x=a)`	Limit of f as $x \to a$
`solve(eqn, x)`	Solve equation for x (exact)
`fsolve(eqn, x)`	Solve equation numerically
`dsolve(ode, y(x))`	Solve ODE
`simplify(expr)`	Simplify expression
`expand(expr)`	Expand expression
`factor(expr)`	Factor expression
`evalf(expr)`	Evaluate to floating point
`plot(f, x=a..b)`	Plot f for $x \in [a, b]$

4.1. MCQs: Maple

Q18. In Maple, which command computes the **definite integral** $\int_0^{\pi} \sin(x)\,dx$?

(A) `int(sin(x), x, 0, Pi)`

(B) `int(sin(x), x=0..Pi)`

(C) `integrate(sin(x), [x, 0, pi])`

(D) `quad(sin, 0, Pi)`

Correct Answer: (B) `int(sin(x), x=0..Pi)`
In Maple, the `int` command uses the syntax `int(f, x=a..b)` for definite integrals. Note that Maple uses `Pi` (capital P) for π. The result is $\int_0^{\pi} \sin x\,dx = [-\cos x]_0^{\pi} = 2$.

Q19. What is the Maple command to **solve the ODE** $y'' + y = 0$?

(A) `solve(diff(y(x),x,x) + y(x) = 0, y(x))`

(B) `dsolve(diff(y(x),x,x) + y(x) = 0, y(x))`

(C) `odesolve(y'' + y = 0)`

(D) `int(diff(y,x,x) + y, x)`

Correct Answer: (B) `dsolve(diff(y(x),x,x) + y(x) = 0, y(x))`
`dsolve` is Maple's ODE solver. The second derivative is `diff(y(x),x,x)` (differentiate w.r.t. x twice). The general solution is $y(x) = C_1 \cos x + C_2 \sin x$.

Q20. In Maple, which command **factors** the expression $x^2 - 5x + 6$?

(A) `simplify(x^2 - 5*x + 6)`

(B) `expand(x^2 - 5*x + 6)`

(C) `factor(x^2 - 5*x + 6)`

(D) `collect(x^2 - 5*x + 6, x)`

Correct Answer: (C) `factor(x^2 - 5*x + 6)`
`factor` factorizes an algebraic expression. The result is `(x-2)*(x-3)`. `expand` does the reverse (distributes products), and `simplify` applies general simplification rules.

Q21. In Maple, what does `evalf(Pi, 20)` compute?

(A) π as a fraction with denominator 20

(B) 20π

(C) π evaluated to 20 significant digits

(D) π rounded to 20 decimal places (always)

Correct Answer: (C) π evaluated to 20 significant digits
`evalf(expr, n)` evaluates `expr` numerically to `n` significant digits. Maple uses exact symbolic arithmetic by default; `evalf` converts to floating-point. `evalf(Pi, 20)` gives 3.1415926535897932385.

5. Mathematica: Wolfram Language

Mathematica (Wolfram Research) features:

- **Notebook interface:** Input/output cells (`.nb` files)
- **Wolfram Language:** Functional, rule-based, symbolic paradigm
- **Built-in functions:** Start with capital letters (e.g., `Integrate`, `Plot`)
- **Pattern matching:** Powerful rewriting rules via `_` (Blank)
- **Exact arithmetic:** Arbitrary precision integers, rationals, surds
- **Kernel + Front End:** Kernel does computations; Front End renders notebooks

Key Commands & Syntax

Essential Mathematica Commands:

Command	Description
`D[f, x]`	Partial/ordinary derivative w.r.t. x
`Integrate[f, x]`	Indefinite integral
`Integrate[f, {x,a,b}]`	Definite integral $\int_a^b f\,dx$
`Limit[f, x->a]`	Limit as $x \to a$
`Solve[eqn, x]`	Exact solutions
`NSolve[eqn, x]`	Numerical solutions
`DSolve[ode, y, x]`	Symbolic ODE solver
`Simplify[expr]`	Simplify expression
`FullSimplify[expr]`	More aggressive simplification
`Expand[expr]`	Expand products/powers
`Factor[expr]`	Factor expression
`N[expr, n]`	Numerical evaluation to n digits
`Plot[f, {x,a,b}]`	2D plot
`Plot3D[f, {x,a,b},{y,c,d}]`	3D surface plot

5.1. MCQs: Mathematica

Q22. In Mathematica, how do you compute $\dfrac{d^2}{dx^2}\sin(x^2)$?

(A) `D[Sin[x^2], {x, 2}]`

(B) `Diff[sin(x^2), x, 2]`

(C) `diff(sin(x^2), x$2)`

(D) `Der[Sin[x^2], x, x]`

Correct Answer: (A) `D[Sin[x^2], {x, 2}]`
In Mathematica, `D[f, {x, n}]` computes the n-th derivative of f with respect to x. Single derivative: `D[f, x]`. Note: Mathematica uses square brackets `[]` for all function calls.

Q23. In Mathematica, which is the correct syntax for **definite integration** $\int_0^1 x^2\,dx$?

(A) `Integrate[x^2, 0, 1]`

(B) `Integrate[x^2, {x, 0, 1}]`

(C) `int(x^2, x=0..1)`

(D) `Int[x^2, x, 0, 1]`

Correct Answer: (B) `Integrate[x^2, {x, 0, 1}]`
Mathematica's definite integral syntax is `Integrate[f, {x, a, b}]` where `{x, a, b}` specifies variable, lower and upper limits. Result: $\int_0^1 x^2\,dx = \frac{1}{3}$ (returned exactly as `1/3`).

Q24. In Mathematica, built-in function names:

(A) Start with a lowercase letter

(B) Start with an uppercase letter

(C) Are all in uppercase

(D) Are all in lowercase

Correct Answer: (B) Start with an uppercase letter
All built-in Mathematica/Wolfram Language functions begin with a capital letter, e.g., `Plot`, `Solve`, `Factor`, `Integrate`. User-defined variables/functions conventionally use lowercase to avoid conflict.

Q25. In Mathematica, the `/.` operator (ReplaceAll) is used for:

(A) Division

(B) Pattern-based substitution

(C) Floating-point evaluation

(D) Module definition

Correct Answer: (B) Pattern-based substitution
The `/.` (ReplaceAll) operator applies transformation rules to an expression. Example: `x^2 + x /. x -> 3` gives `12`. It is central to Mathematica's rule-based programming paradigm.

FPSC Exam Tip — Mathematica Syntax Rules:

- Square brackets `[]` for all function arguments (not parentheses)
- Curly braces `{}` for lists and ranges
- `==` for equations (not `=` which is assignment)
- `:=` for delayed assignment (evaluated only when called)
- `Pi`, `E`, `I` are built-in constants (capitalized)

6. Python for Mathematics: NumPy, SciPy, and SymPy

Python Scientific Stack

Python's mathematical power comes from its libraries:

- **NumPy:** N-dimensional array object, linear algebra, FFT, random numbers
- **SciPy:** Built on NumPy; integration, optimization, statistics, signal pro-

cessing, ODE solving

- **SymPy:** Symbolic mathematics (differentiation, integration, equation solving, simplification)
- **Matplotlib:** 2D/3D plotting
- **Pandas:** Data structures and analysis
- **Jupyter Notebook:** Interactive computing environment (like Mathematica notebooks)

Key Commands & Syntax

Python Mathematical Commands:

Command (with import)	Description
`np.array([1,2,3])`	Create NumPy array
`np.zeros((m,n))`	$m \times n$ zero array
`np.eye(n)`	$n \times n$ identity matrix
`np.dot(A, B)`	Matrix multiplication
`np.linalg.det(A)`	Determinant
`np.linalg.inv(A)`	Matrix inverse
`np.linalg.eig(A)`	Eigenvalues and eigenvectors
`scipy.integrate.quad(f,a,b)`	Definite integral numerically
`scipy.optimize.fsolve`	Solve nonlinear equations
`sp.diff(f, x)`	SymPy: differentiate
`sp.integrate(f, x)`	SymPy: integrate
`sp.solve(eqn, x)`	SymPy: solve equation
`sp.simplify(expr)`	SymPy: simplify
`plt.plot(x, y)`	Matplotlib: 2D plot

6.1. MCQs: Python for Mathematics

Q26. Which Python library provides the **array** object optimized for numerical computation?

(A) SymPy

(B) Matplotlib

(C) NumPy

(D) Pandas

Correct Answer: (C) NumPy
NumPy (Numerical Python) provides the fundamental `ndarray` object for efficient numerical computation. It supports vectorized operations, broadcasting, and interfaces with linear algebra routines.

Q27. In Python with **SymPy**, how do you symbolically differentiate $f(x) = x^3 \sin(x)$?

(A) `np.diff(x**3 * np.sin(x))`

(B) `sp.diff(x**3 * sp.sin(x), x)`

(C) `derivative(x**3 * sin(x), x)`

(D) `d/dx(x**3 * sin(x))`

Correct Answer: (B) `sp.diff(x**3 * sp.sin(x), x)`
In SymPy (imported as `sp` or `from sympy import *`), `sp.diff(expr, var)` computes symbolic derivatives. The result is $3x^2 \sin x + x^3 \cos x$. Note: `x` must be declared as a SymPy symbol: `x = sp.symbols('x')`.

Q28. What does `scipy.integrate.quad(f, 0, 1)` return in Python?

(A) The symbolic antiderivative of f

(B) A tuple of (numerical integral value, error estimate)

(C) A list of sample points

(D) The definite integral as a fraction

Correct Answer: (B) A tuple of (numerical integral value, error estimate)
`scipy.integrate.quad` performs numerical quadrature (Gaussian quadrature). It returns a tuple `(value, error)` where `value` is the approximate integral and `error` is an estimate of the absolute error.

Q29. In Python, what is the difference between `**` and `^` operators?

(A) Both compute exponentiation

(B) `**` is exponentiation; `^` is bitwise XOR

(C) `^` is exponentiation; `**` is bitwise XOR

(D) `**` is matrix power; `^` is scalar power

Correct Answer: (B) `**` is exponentiation; `^` is bitwise XOR
In Python, `2**3 = 8` (exponentiation). The `^` operator is bitwise XOR (`2^3 = 1`). This is a common source of bugs for mathematicians transitioning to Python from MATLAB (where `^` is exponentiation).

Q30. In Python with NumPy, what is the result of `np.linspace(0, 1, 5)`?

(A) `[0, 0.25, 0.5, 0.75]`

(B) `[0, 0.25, 0.5, 0.75, 1.0]`

(C) `[0, 1, 2, 3, 4]`

(D) `[0.2, 0.4, 0.6, 0.8, 1.0]`

Correct Answer: (B) `[0, 0.25, 0.5, 0.75, 1.0]`
`np.linspace(start, stop, num)` returns `num` evenly spaced values from `start` to `stop` (inclusive). So `np.linspace(0, 1, 5)` = `[0, 0.25, 0.5, 0.75, 1.0]`. This is analogous to MATLAB's `linspace(0,1,5)`.

FPSC Exam Tip — Python vs. MATLAB:

Task	MATLAB	Python (NumPy)
Array indexing	Starts at 1	Starts at 0
Exponentiation	`^` or `.^`	`**`
Matrix multiply	`*`	`@` or `np.dot()`
Linspace	`linspace(a,b,n)`	`np.linspace(a,b,n)`
Plotting	`plot(x,y)`	`plt.plot(x,y)`
Comments	`%`	`#`

7. R Language: Statistical Computing

R is a free, open-source language for statistical computing and graphics:

- Developed by Ross Ihaka and Robert Gentleman (1993); maintained by the R Foundation
- Indexing: **1-based** (like MATLAB)
- Assignment: `x <- 5` or `x = 5`
- Packages: 18,000+ on CRAN (Comprehensive R Archive Network)
- Vectors are the fundamental data type (no scalars)
- **RStudio:** Popular IDE for R

7.1. MCQs: R Language

Q31. In R, which function computes the **mean** of a vector `x`?

(A) `average(x)`

(B) `mean(x)`

(C) `avg(x)`

(D) `arithmetic(x)`

Correct Answer: (B) `mean(x)`
R uses `mean(x)` for arithmetic mean, `median(x)` for median, `var(x)` for variance, and `sd(x)` for standard deviation. There is no built-in `average()` function in base R.

Q32. In R, what does `c(1, 2, 3, 4)` create?

(A) A matrix of size 1×4

(B) A list of 4 elements

(C) A vector containing 1, 2, 3, 4

(D) A character string

Correct Answer: (C) A vector containing 1, 2, 3, 4
`c()` (combine) is R's primary function to create vectors. R's vectors are the fundamental data structure. A 1×4 matrix would be `matrix(c(1,2,3,4), nrow=1)`.

Q33. Which R function fits a **linear regression** model?

(A) `regression(y ~ x)`

(B) `lm(y ~ x)`

(C) `glm(y, x)`

(D) `linfit(y, x)`

Correct Answer: (B) `lm(y ~ x)`
`lm()` (linear model) is R's function for fitting linear regression. The tilde `~` notation defines the model formula: `lm(y ~ x1 + x2)` fits $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$. `glm()` is for generalized linear models.

8. GNU Octave and Scilab

GNU Octave is a free, open-source alternative to MATLAB:

- Syntax is largely compatible with MATLAB (most `.m` scripts run unchanged)
- Key differences: Octave allows `!` for `not`, `#` for comments, `++` increment
- **Free:** GNU General Public License (GPL)
- Uses the same matrix-based computation model as MATLAB
- Octave-Forge: Community packages (like MATLAB's toolboxes)

Scilab is another free alternative (by ESI Group):

- Uses `.sce` (scripts) and `.sci` (function files)
- **Key difference from MATLAB:** Uses `^` for element-wise power (no `.^` needed for scalars); string concatenation with `+`
- ATOMS: Scilab's package manager
- Has its own IDE called **Xcos** (for block diagrams, similar to Simulink)
- Not directly compatible with MATLAB syntax (unlike Octave)

8.1. MCQs: GNU Octave and Scilab

Q34. GNU Octave is primarily designed as a free alternative to:

(A) Mathematica

(B) Maple

(C) MATLAB

(D) R

Correct Answer: (C) MATLAB
GNU Octave was specifically designed as a high-level language mostly compatible with MATLAB. The goal is that most MATLAB scripts run without modification in Octave.

Q35. Which of the following is a feature of GNU Octave that is **NOT** present in standard MATLAB?

(A) Matrix operations

(B) The `#` character as a comment symbol

(C) The `plot` function

(D) The `for` loop

Correct Answer: (B) The # character as a comment symbol
Octave allows both `%` (MATLAB-style) and `#` for comments. Standard MATLAB only uses `%`. Octave also permits the `++` and `--` increment operators, which MATLAB does not support.

Q36. The block diagram / visual programming tool in Scilab (analogous to MATLAB Simulink) is:

(A) Scicos / Xcos

(B) Blockset

(C) LabView

(D) Simulink

Correct Answer: (A) Scicos / Xcos
Xcos (formerly Scicos) is Scilab's graphical simulation environment for modelling dynamic systems using block diagrams, analogous to MATLAB's Simulink.

9. Numerical Methods in Computing Tools

All major mathematical computing tools support numerical methods:

Method	MATLAB	Python (SciPy)
Root finding	`fzero, fsolve`	`scipy.optimize.fsolve`
Numerical ODE	`ode45, ode23`	`scipy.integrate.odeint`
Numerical integral	`integral, quad`	`scipy.integrate.quad`
Optimization	`fminbnd, fminsearch`	`scipy.optimize.minimize`
Interpolation	`interp1, interp2`	`scipy.interpolate`
FFT	`fft, ifft`	`np.fft.fft`

9.1. MCQs: Numerical Methods

Q37. Which MATLAB solver is commonly used for **non-stiff ODEs** using a Runge–Kutta method?

(A) `ode15s`

(B) `ode45`

(C) `ode23t`

(D) `pdepe`

Correct Answer: (B) `ode45`
`ode45` implements the Dormand-Prince Runge-Kutta (4,5) method and is MATLAB's recommended solver for non-stiff ODEs. For stiff systems, `ode15s` (based on NDFs) is preferred. `pdepe` solves parabolic-elliptic PDEs.

Q38. In MATLAB, `fft(x)` computes:

(A) Fast Fourier Transform of vector `x`

(B) Fast fractional transform

(C) Finite field transform

(D) Filtered Fourier transform

Correct Answer: (A) Fast Fourier Transform of vector `x`
`fft` computes the Discrete Fourier Transform (DFT) using the Fast Fourier Transform algorithm. `ifft` is the inverse. These are extensively used in signal processing, image processing, and solving PDEs spectrally.

Q39. In Python (SciPy), which function numerically solves an initial-value ODE $\dot{y} = f(t, y)$?

(A) `scipy.integrate.odeint` or `solve_ivp`

(B) `scipy.integrate.quad`

(C) `numpy.linalg.solve`

(D) `scipy.optimize.fsolve`

Correct Answer: (A) `scipy.integrate.odeint` or `solve_ivp`
Both `odeint` (older, wraps LSODA) and `solve_ivp` (newer, more flexible) solve initial-value ODEs in SciPy. `quad` is for numerical integration, `linalg.solve` for linear systems, and `fsolve` for algebraic equations.

10. Comparative Overview and Miscellaneous MCQs

Key Commands & Syntax

Quick Comparison Table:

Feature	MATLAB	Maple	Mathematica	Python
Developer	MathWorks	Maplesoft	Wolfram	PSF
Primary Use	Numerical	Symbolic	Both	General
File Extension	`.m`	`.mw`	`.nb`	`.py`
Cost	Proprietary	Proprietary	Proprietary	Free
Indexing	1-based	1-based	1-based	0-based
Comment Symbol	`%`	`#`	`(*...*)`	`#`

10.1. MCQs: Comparison and Applications

Q40. Which tool is best suited for **large-scale statistical analysis** and has the most packages for statistical methods?

(A) MATLAB

(B) Mathematica

(C) R

(D) Scilab

Correct Answer: (C) R
R was designed specifically for statistical computing and has over 18,000 packages on CRAN covering virtually every statistical method. It is the de facto standard in academia and industry for statistics, bioinformatics, and data science.

Q41. In Mathematica, comments are written using:

(A) `%` symbol

(B) `#` symbol

(C) `(* comment *)`

(D) `// comment`

Correct Answer: (C) `(* comment *)`
Mathematica uses `(* ... *)` for comments (can be multi-line). MATLAB and R use `%` and `#` respectively. Python uses `#` for single-line comments and `"""..."""` for docstrings.

Q42. Which Python library is used for **2D and 3D plotting and visualization**?

(A) NumPy

(B) SymPy

(C) SciPy

(D) Matplotlib

Correct Answer: (D) Matplotlib
Matplotlib is Python's primary plotting library, providing `pyplot` for MATLAB-like plotting. Seaborn, Plotly, and Bokeh are built on top of or alongside Matplotlib for more advanced visualizations.

Q43. The **SageMath** platform is best described as:

(A) A proprietary CAS by IBM

(B) A free open-source mathematics software integrating many packages including Python, Maxima, and R

(C) A statistical package for social sciences

(D) A MATLAB toolbox for symbolic computation

Correct Answer: (B) A free open-source mathematics software integrating many packages
SageMath (formerly SAGE) is a free, open-source mathematics software system

that integrates NumPy, SciPy, Matplotlib, SymPy, Maxima, GAP, FLINT, R, and many other packages under a common Python-based interface.

Q44. In MATLAB, the **Symbolic Math Toolbox** allows MATLAB to perform:

(A) GPU parallel computing

(B) Symbolic computation using MuPAD or Maple kernel

(C) Deep learning tasks

(D) Real-time data acquisition

Correct Answer: (B) Symbolic computation using MuPAD or Maple kernel
MATLAB's Symbolic Math Toolbox (an add-on) enables symbolic computation. Historically it used a MuPAD kernel; more recent versions use a Maple kernel under the hood. It adds `syms`, `sym`, `diff`, `int`, `solve` etc. as symbolic functions in MATLAB.

Q45. Which of the following MATLAB commands creates a **3D surface plot**?

(A) `plot3(x,y,z)`

(B) `surf(X,Y,Z)`

(C) `mesh3(X,Y,Z)`

(D) `surface(x,y,z)`

Correct Answer: (B) `surf(X,Y,Z)`
`surf(X,Y,Z)` creates a shaded 3D surface plot. `mesh(X,Y,Z)` creates a wireframe surface. `plot3(x,y,z)` plots a 3D line/curve (not a surface). `X` and `Y` are grid matrices (from `meshgrid`).

Q46. In Python, the `@` operator (introduced in Python 3.5) is used for:

(A) Decorator syntax only

(B) Matrix multiplication (PEP 465)

(C) String formatting

(D) Module import

Correct Answer: (B) Matrix multiplication (PEP 465)
Since Python 3.5, the `@` operator performs matrix multiplication for NumPy arrays (and any object implementing `__matmul__`). So `A @ B` is equivalent to `np.matmul(A, B)` or `np.dot(A, B)` for 2D arrays.

Q47. What is the primary purpose of **Jupyter Notebook**?

(A) A word processor for mathematical documents

(B) An interactive web-based computing environment supporting Python, R, Julia and more

(C) A graphical user interface for MATLAB

(D) A database management system

Correct Answer: (B) An interactive web-based computing environment
Jupyter Notebook (from Project Jupyter) is a web-based interactive environment that allows live code, equations, visualizations, and narrative text in a single document. It supports Python, R, Julia, and many other kernels via the open notebook format (`.ipynb`).

Final FPSC Exam Tips — Computing Tools:

1. **Free vs Proprietary:** Python, R, GNU Octave, Scilab, SageMath, Maxima = free. MATLAB, Maple, Mathematica = paid.
2. **File extensions:** MATLAB `.m`, Mathematica `.nb`, Maple `.mw`, Python `.py`, R `.R`, Jupyter `.ipynb`
3. **Indexing:** MATLAB/R/Maple/Mathematica = 1-based; Python/C/Java = 0-based
4. **ODE solvers:** MATLAB `ode45`; Python `solve_ivp`; Maple `dsolve`; Mathematica `DSolve` (symbolic) / `NDSolve` (numerical)
5. **Symbolic vs Numerical:** Maple and Mathematica are primarily symbolic; MATLAB and Octave primarily numerical; Python does both via libraries
6. **Wolfram Alpha:** Online CAS by Wolfram Research; uses same Wolfram Language as Mathematica

Answer Key Summary

Answer Key

Q1. (C)	Q12. (C)	Q22. (B)	Q32. (C)
Q2. (C)	Q13. (B)	Q23. (A)	Q33. (A)
Q3. (B)	Q14. (C)	Q24. (B)	Q34. (B)
Q4. (C)	Q15. (B)	Q25. (B)	Q35. (A)
Q5. (C)	Q16. (A)	Q26. (B)	Q36. (B)
Q6. (B)	Q17. (B)	Q27. (C)	Q37. (C)
Q7. (B)	Q18. (B)	Q28. (B)	Q38. (B)
Q8. (B)	Q19. (B)	Q29. (C)	Q39. (B)
Q9. (C)	Q20. (C)	Q30. (A)	Q40. (B)
Q10. (B)	Q21. (B)	Q31. (B)	
Q11. (B)			

Best of Luck for Your FPSC/PPSC/AJKPSC Examination!

Prepared by Zamir Hussain — Lecturer Mathematics, University of Wah